Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Томский государственный университет систем управления и радиоэлектроники»

Факультет систем управления (ФСУ)

Кафедра автоматизированных систем управления (АСУ)

Обработка данных на Kaggle. Доступ к удаленным ресурсам.

Отчет по лабораторной работе №2 по дисциплине
«Разработка веб-сервисов для научных и прикладных задач»

Обучающийся гр. 431-3
_____ Андреев.Д.П.
«___» _____ 2022 г.

Проверил: Кандидат технических наук
_____ Суханов А.Я.
«___» _____ 2022 г.

Томск 2021

Оглавление

1. Введение 3
2. Ход выполнения работы 3
2.1. Работа с Kaggle 3
2.2. Анализ данных 4
2.2. Работа с графиками 6
3. Работа с индивидуальным заданием 9
3.1. Решение варианта 9
4. Изучение возможностей доступа к API сервиса data.gov.ru 12
5. Вывод 14
6. Приложение 15
7. Дополнительные Задания 8

1. Введение

2. Ход выполнения работы

2.1. Работа с Kaggle

В пункте 1.7.2 методички есть примеры работы с датасетами. Первым делом нужно выбрать датасет из списка.

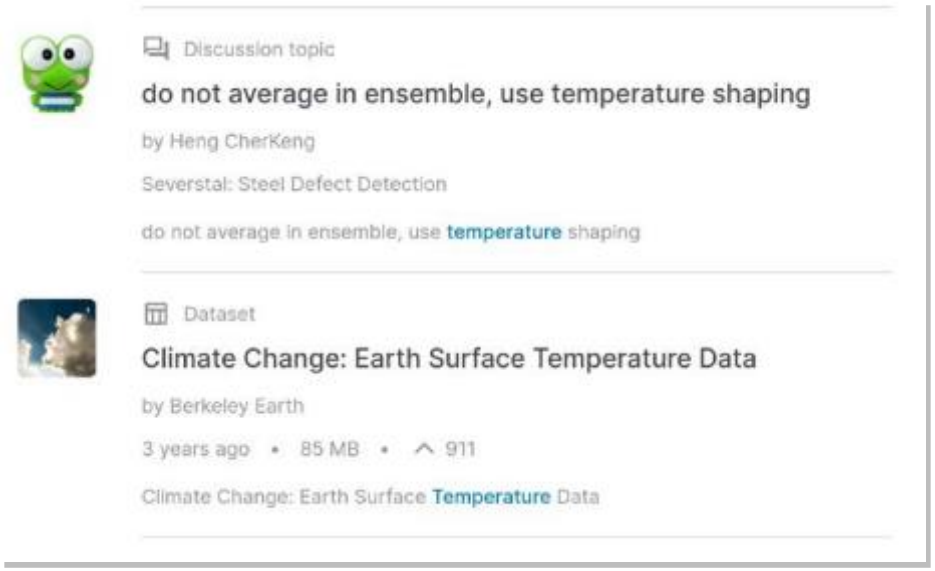


Рисунок 2.1 – выбор датасета по температуре

Первым делом выведим данные из датасета (Код в приложении):

```
/kaggle/input/climate-change-earth-surface-temperature-data/GlobalTemperatures.csv
/kaggle/input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByState.csv
/kaggle/input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCountry.csv
/kaggle/input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCity.csv
/kaggle/input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByMajorCity.csv
/kaggle/input/car-sales/Car_sales.csv
                dt  AverageTemperature  AverageTemperatureUncertainty  \
0       1743-11-01               4.384                          2.294
1       1743-12-01                 NaN                            NaN
2       1744-01-01                 NaN                            NaN
3       1744-02-01                 NaN                            NaN
4       1744-03-01                 NaN                            NaN
...            ...                 ...                            ...
577457  2013-05-01              19.059                          1.022
577458  2013-06-01              17.613                          0.473
577459  2013-07-01              17.000                          0.453
577460  2013-08-01              19.759                          0.717
577461  2013-09-01                 NaN                            NaN

         Country
0          Åland
1          Åland
2          Åland
3          Åland
4          Åland
...          ...
577457  Zimbabwe
577458  Zimbabwe
577459  Zimbabwe
577460  Zimbabwe
577461  Zimbabwe

[577462 rows x 4 columns]
                dt  AverageTemperature  AverageTemperatureUncertainty  \
0        1743-11-01               6.068                          1.737
1        1743-12-01                 NaN                            NaN
2        1744-01-01                 NaN                            NaN
3        1744-02-01                 NaN                            NaN
4        1744-03-01                 NaN                            NaN
...             ...                 ...                            ...
8599207  2013-05-01              11.464                          0.236
8599208  2013-06-01              15.043                          0.261
8599209  2013-07-01              18.775                          0.193
8599210  2013-08-01              18.025                          0.298
8599211  2013-09-01                 NaN                            NaN

           City      Country Latitude Longitude
0         Århus      Denmark   57.05N    10.33E
1         Århus      Denmark   57.05N    10.33E
2         Århus      Denmark   57.05N    10.33E
3         Århus      Denmark   57.05N    10.33E
4         Århus      Denmark   57.05N    10.33E
...         ...          ...      ...       ...
8599207  Zwolle  Netherlands   52.24N     5.26E
8599208  Zwolle  Netherlands   52.24N     5.26E
8599209  Zwolle  Netherlands   52.24N     5.26E
8599210  Zwolle  Netherlands   52.24N     5.26E
8599211  Zwolle  Netherlands   52.24N     5.26E

[8599212 rows x 7 columns]
```

Рисунок 2.2 – результат работы кода с выводом данных датасета

2.2. Анализ данных

Как мы видим, что данные состоят из списка данных, теперь можно приступать к анализу предоставленных данных, например, можно будет узнать различные характеристики температур в городах.

```
       AverageTemperature  AverageTemperatureUncertainty
count        8.235082e+06                   8.235082e+06
mean         1.672743e+01                   1.028575e+00
std          1.035344e+01                   1.129733e+00
min         -4.270400e+01                   3.400000e-02
25%          1.029900e+01                   3.370000e-01
50%          1.883100e+01                   5.910000e-01
75%          2.521000e+01                   1.349000e+00
max          3.965100e+01                   1.539600e+01
```

Рисунок 2.3 – результат работы кода с анализом датасета

Давайте узнаем список всех стран, которые есть в данном датасете (Код в приложении).

```
['Denmark', 'Turkey', 'Kazakhstan', 'China', 'Spain', 'Germany', 'Nigeria', 'Iran', 'Russia', 'C
anada', "Côte D'Ivoire", 'United Kingdom', 'Saudi Arabia', 'Japan', 'United States', 'India', 'B
enin', 'United Arab Emirates', 'Mexico', 'Venezuela', 'Ghana', 'Ethiopia', 'Australia', 'Yemen',
'Indonesia', 'Morocco', 'Pakistan', 'France', 'Libya', 'Burma', 'Brazil', 'South Africa', 'Syri
a', 'Egypt', 'Algeria', 'Netherlands', 'Malaysia', 'Portugal', 'Ecuador', 'Italy', 'Uzbekistan',
'Philippines', 'Madagascar', 'Chile', 'Belgium', 'El Salvador', 'Romania', 'Peru', 'Colombia',
'Tanzania', 'Tunisia', 'Turkmenistan', 'Israel', 'Eritrea', 'Paraguay', 'Greece', 'New Zealand',
'Vietnam', 'Cameroon', 'Iraq', 'Afghanistan', 'Argentina', 'Azerbaijan', 'Moldova', 'Mali', 'Con
go (Democratic Republic Of The)', 'Thailand', 'Central African Republic', 'Bosnia And Herzegovin
a', 'Bangladesh', 'Switzerland', 'Equatorial Guinea', 'Cuba', 'Lebanon', 'Mozambique', 'Serbia',
'Angola', 'Somalia', 'Norway', 'Nepal', 'Poland', 'Ukraine', 'Guinea Bissau', 'Malawi', 'Burkina
Faso', 'Slovakia', 'Congo', 'Belarus', 'Gambia', 'Czech Republic', 'Hungary', 'Burundi', 'Zimbab
we', 'Bulgaria', 'Haiti', 'Puerto Rico', 'Sri Lanka', 'Nicaragua', 'Zambia', 'Honduras', 'Taiwa
n', 'Bolivia', 'Guinea', 'Ireland', 'Senegal', 'Latvia', 'Qatar', 'Albania', 'Tajikistan', 'Keny
a', 'Guatemala', 'Finland', 'Sierra Leone', 'Sweden', 'Botswana', 'Guyana', 'Austria', 'Uganda',
'Armenia', 'Dominican Republic', 'Jordan', 'Djibouti', 'Sudan', 'Lithuania', 'Rwanda', 'Jamaic
a', 'Togo', 'Macedonia', 'Cyprus', 'Gabon', 'Slovenia', 'Bahrain', 'Swaziland', 'Niger', 'Lesoth
o', 'Liberia', 'Uruguay', 'Chad', 'Bahamas', 'Mauritania', 'Panama', 'Suriname', 'Cambodia', 'Mo
ntenegro', 'Mauritius', 'Papua New Guinea', 'Iceland', 'Croatia', 'Reunion', 'Oman', 'Costa Ric
a', 'South Korea', 'Hong Kong', 'Singapore', 'Estonia', 'Georgia', 'Mongolia', 'Laos', 'Namibi
a']
```

Рисунок 2.4 – результат работы кода с выводом всех стран из датасета

С помощью Python можно также узнать все данные связанные с Россией(Код в приложении).

```
['dt', 'AverageTemperature', 'AverageTemperatureUncertainty', 'City', 'Country', 'Latitude', 'Lo
ngitude']
['Russia']
                 dt  AverageTemperature  AverageTemperatureUncertainty  \
30064    1820-08-01              13.221                          2.706
30065    1820-09-01               7.744                          1.756
30066    1820-10-01              -1.771                          1.837
30067    1820-11-01             -11.387                          2.218
30068    1820-12-01             -19.725                          2.549
...             ...                 ...                            ...
8578952  2013-05-01              10.688                          0.349
8578953  2013-06-01              16.522                          0.398
8578954  2013-07-01              17.782                          0.555
8578955  2013-08-01              16.000                          0.412
8578956  2013-09-01                 NaN                            NaN

             City Country Latitude Longitude
30064      Abakan  Russia   53.84N    91.36E
30065      Abakan  Russia   53.84N    91.36E
30066      Abakan  Russia   53.84N    91.36E
30067      Abakan  Russia   53.84N    91.36E
30068      Abakan  Russia   53.84N    91.36E
...           ...     ...      ...       ...
8578952  Zlatoust  Russia   55.45N    59.53E
8578953  Zlatoust  Russia   55.45N    59.53E
8578954  Zlatoust  Russia   55.45N    59.53E
8578955  Zlatoust  Russia   55.45N    59.53E
8578956  Zlatoust  Russia   55.45N    59.53E

[461234 rows x 7 columns]
```

Рисунок 2.5 – результат работы кода с выводом всех данных связанные с Россией

Также с помощью фильтра можно вывести примерные значения стран. Например, при использовании слова «United» - мы получаем значения ['United Kingdom', 'United States', 'United Arab Emirates']. Кроме этого можно отсортировать несколько стран, перечислив их через запятую.

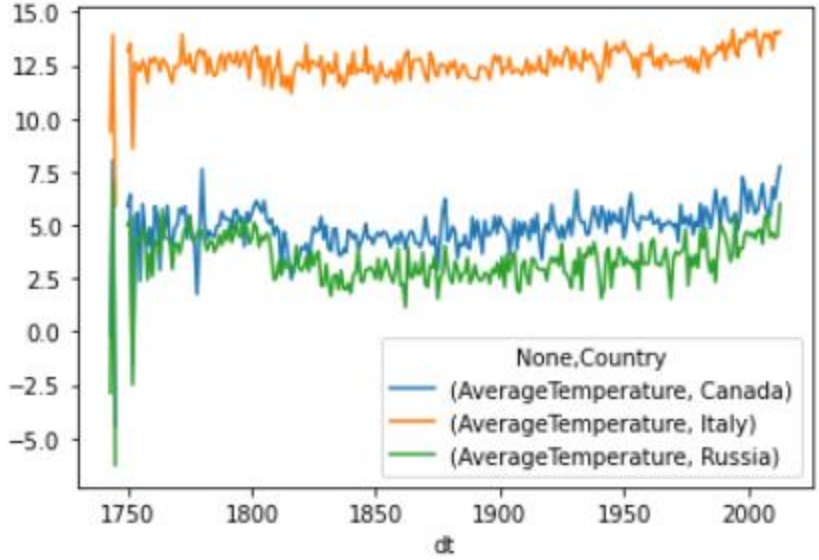
```
['Country', 'dt', 'AverageTemperature', 'AverageTemperatureUncertainty']
      Country    dt  AverageTemperature  AverageTemperatureUncertainty
5231   Canada  1743           -0.206687                       1.839375
5232   Canada  1744            8.046000                       1.744164
5233   Canada  1745           -4.417203                       1.836437
5234   Canada  1746                 NaN                            NaN
5235   Canada  1747                 NaN                            NaN
...       ...   ...                 ...                            ...
24909  Russia  2009            4.449978                       0.307108
24910  Russia  2010            4.640022                       0.328615
24911  Russia  2011            4.386322                       0.320338
24912  Russia  2012            4.501149                       0.536374
24913  Russia  2013            5.999847                       0.721382

[813 rows x 4 columns]
[('AverageTemperature', 'Canada'), ('AverageTemperature', 'Italy'), ('AverageTemperature', 'Russ
ia'), ('AverageTemperatureUncertainty', 'Canada'), ('AverageTemperatureUncertainty', 'Italy'),
('AverageTemperatureUncertainty', 'Russia')]
```

Рисунок 2.6 – результат работы кода с выводом информации для несколько стран

2.2. Работа с графиками

Также мы можем визуализировать данные при помощи графиков. Графики создаются при помощи .plot().



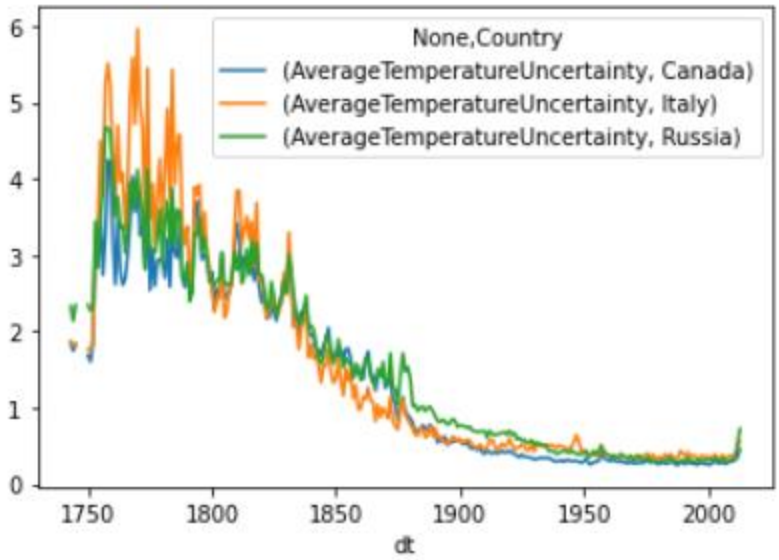


Рисунок 2.7 – результат работы кода с выводом графиков средних температур

Полезный график pairplot в seaborn позволяет посмотреть соотношение данных попарно по отношению друг к другу.

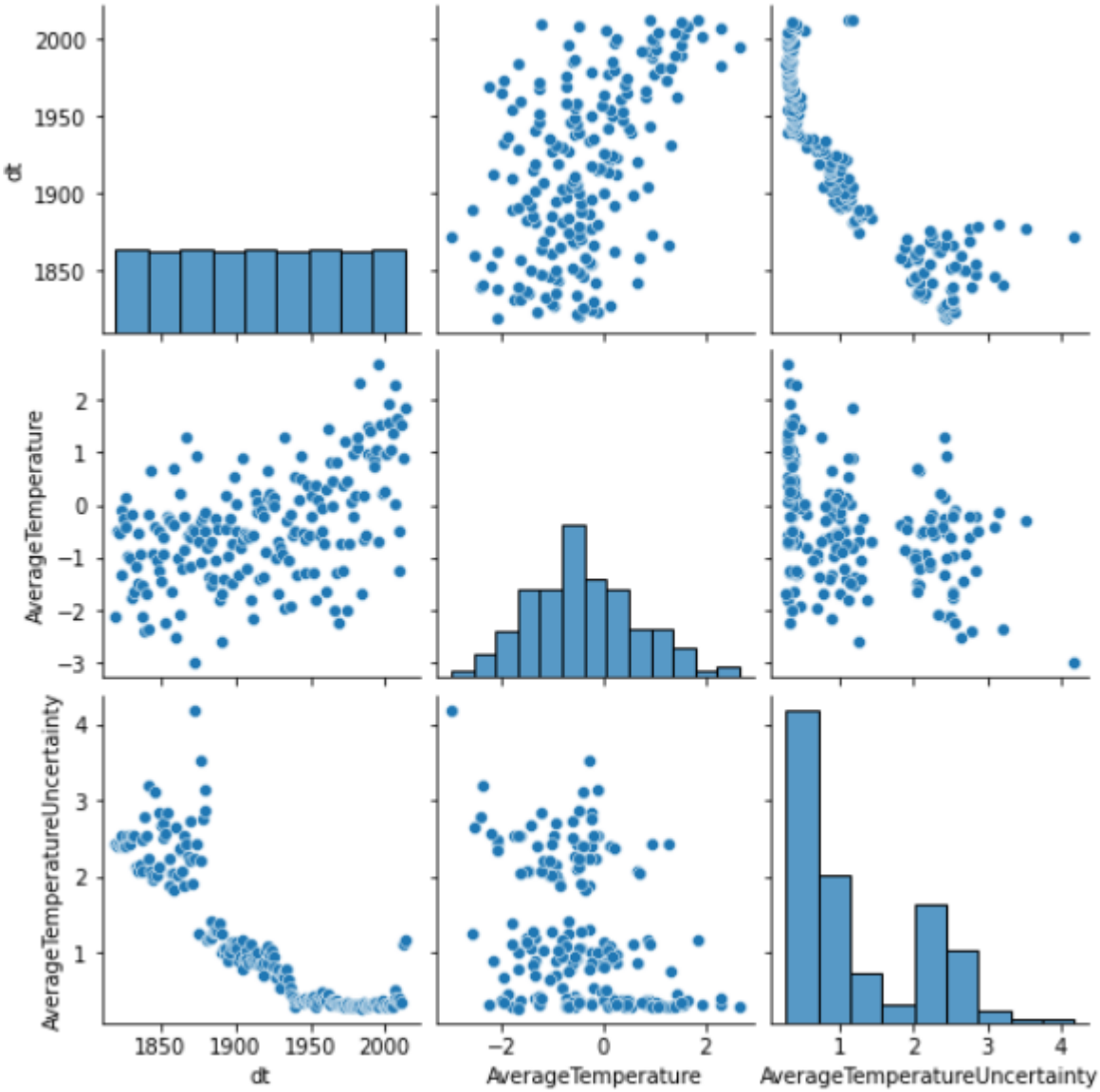


Рисунок 2.8 – результат работы кода с выводом графиков всех данных томска

3. Работа с индивидуальным заданием

Посчитать среднюю температуру по всем годам для Италии, Германии и России (Вывести данные в виде графиков по годам и температурам)

3.1. Решение варианта

Внизу представлены результаты работы в виде рисунков (Рисунки 3.1-3.)

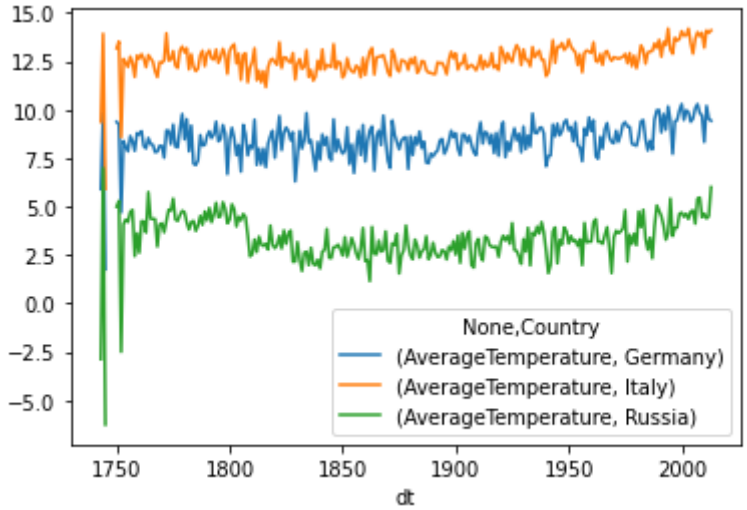


Рисунок 3.1 – результат работы кода с выводом температур стран

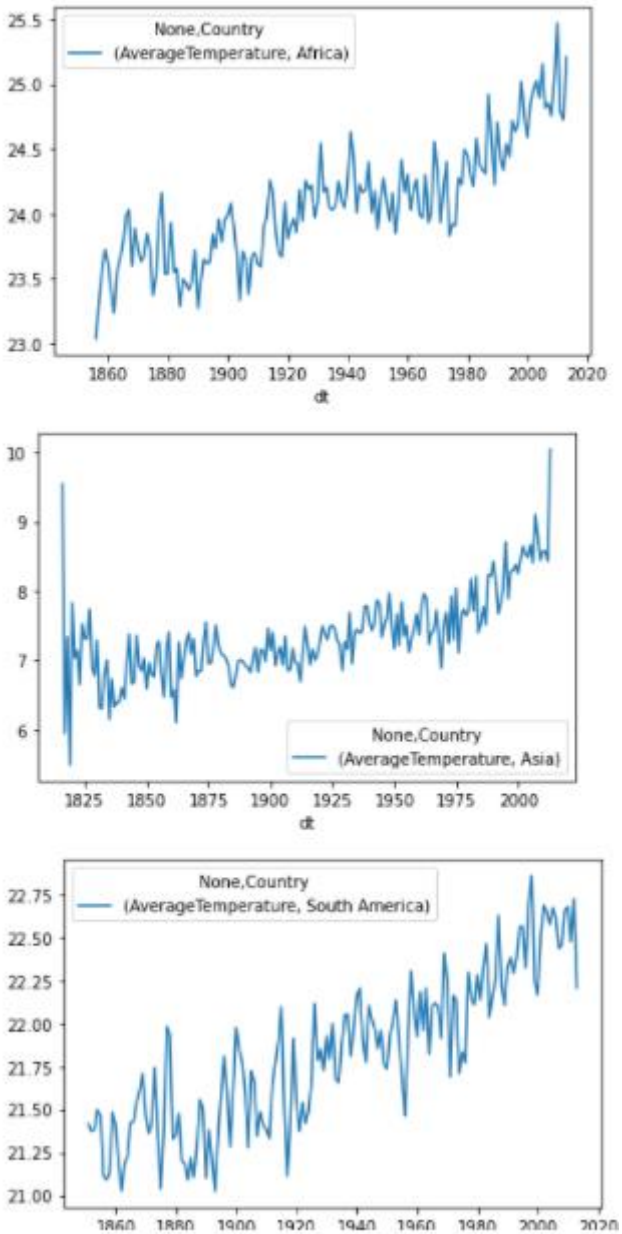


Рисунок 3.2 – результат работы кода с выводом температур континентов

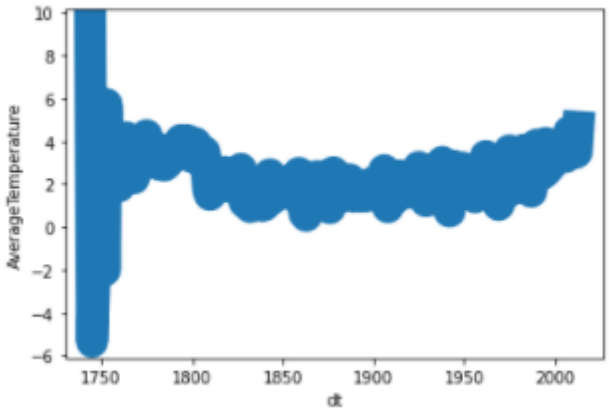


Рисунок 3.2 – результат работы кода сглаживания графика(По двум точкам)

4. Изучение возможностей доступа к API сервиса data.gov.ru

Обычно подобные data.gov.ru сервисы предоставляют API доступ и передачу данных приложению клиенту в форматах json, xml, csv, по протоколу HTTP.

Для получения доступа к API воспользуемся компонентом http.client, позволяющим выполнять HTTP-запросы и получать ответы. Используем основной тип запроса для получения данных с сервера GET, далее указываем URL и протокол с версией. Затем идут заголовки запроса.

```
9996                          7710747640-roznadn
9997                           7710747640-vznmpt
9998  2624035752-perecen-uchregdenii-kulturi-BMOSK
9999                            2645006819-podved

                                              title  organization  \
0        Объем перевозок грузов на морском транспорте.  7.708235e+09
1            Объем переработки грузов в морских портах  7.708235e+09
2     Объем переработки грузов на внутреннем водном ...  7.708235e+09
3     Объем платежей в бюджетную систему Российской ...  7.708235e+09
4     Объем платных услуг населению (оперативные дан...  7.708235e+09
...                                                 ...            ...
9995  Список центров общественного доступа Администр...  8.601042e+09
9996  Розничные продажи алкогольной продукции на душ...  7.710748e+09
9997  Информация о выданных заключениях, необходимых...  7.710748e+09
9998  Перечень учреждений культуры Буденновского мун...  2.624036e+09
9999             Перечень подведомственных организаций  2.645007e+09

                                  organization_name       topic
0        Федеральная служба государственной статистики  Government
1        Федеральная служба государственной статистики  Government
2        Федеральная служба государственной статистики  Government
3        Федеральная служба государственной статистики  Government
4        Федеральная служба государственной статистики  Government
...                                              ...         ...
9995  Департамент информационных технологий и цифров...        None
9996  Федеральная служба по регулированию алкогольно...  Government
9997  Федеральная служба по регулированию алкогольно...  Government
9998  Администрация Буденновского муниципального округа  Government
9999  Администрация Шпаковского муниципального округ...  Government

[10000 rows x 5 columns]
[
  {
    "identifier": "7708234640-fiveasevenasixasevenaone",
    "title": "Объем перевозок грузов на морском транспорте.",
    "organization": "7708234640",
    "organization_name": "Федеральная служба государственной статистики",
```

Рисунок 4.1 – результат работы кода с выводом датасета из data.gov.ru

```
import pandas
fname = '/content/drive/MyDrive/python/dataf/dataset.hdf5'
dt = pandas.read_hdf(fname, key = 'datasets_2019')
print(dt)
```

```
                                   identifier  \
0             7708234640-fiveasevenasixasevenaone
1            7708234640-threeafiveazeroathreeasix
2           7708234640-threeafiveazeroathreeafive
3          7708234640-fourathreeafourafourathree
4         7708234640-fiveasevenasevenaeightaeight
...                                           ...
9995                            8618002968-cod_hmr
9996                            7710747640-roznadn
9997                             7710747640-vznmpt
9998  2624035752-perecen-uchregden11-kultur1-BMOSK
9999                             2645006819-podved

                                              title  organization  \
0        Объем перевозок грузов на морском транспорте.  7.708235e+09
1            Объем переработки грузов в морских портах  7.708235e+09
2    Объем переработки грузов на внутреннем водном ...  7.708235e+09
3    Объем платежей в бюджетную систему Российской ...  7.708235e+09
4    Объем платных услуг населению (оперативные дан...  7.708235e+09
...                                                ...           ...
9995  Список центров общественного доступа Администр...  8.601042e+09
9996  Розничные продажи алкогольной продукции на душ...  7.710748e+09
9997  Информация о выданных заключениях, необходимых...  7.710748e+09
9998  Перечень учреждений культуры Буденновского мун...  2.624036e+09
9999              Перечень подведомственных организаций  2.645007e+09

                                organization_name       topic
0        Федеральная служба государственной статистики  Government
1        Федеральная служба государственной статистики  Government
2        Федеральная служба государственной статистики  Government
3        Федеральная служба государственной статистики  Government
4        Федеральная служба государственной статистики  Government
```

Рисунок 4.2 – результат работы кода с записью данных из датасета из data.gov.ru

Как мы можем видеть, мы смогли сделать парсер на данные.

5. Вывод

С помощью python-а можно решить различные задачи, связанные не только с математикой и с обработкой фото, но и с обработкой дата сетов.

6. Приложение

Вывод данных датасета:

```
import numpy as np # linear algebra
# data processing, CSV file I/O (e.g.
pd.read_csv)
import pandas as pd
# Input data files are available in th
e "../input/" directory.
# For example, running this (by clicki
ng run or pressing Shift+Enter)
# will list all files under the input
directory
import os

for dirname, _, filenames in os.walk('
/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, fi
lename))
# Any results you write to the current
 directory are saved as output.
import pandas as pd

GlobalLandTemperaturesByCity = pd.read
_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCity.csv"
)
GlobalLandTemperaturesByCountry = pd.r
ead_csv("../input/climate-change-
earth-surface-temperature-
data/GlobalLandTemperaturesByCountry.c
sv")
GlobalLandTemperaturesByMajorCity = pd
.read_csv("../input/climate-change-
earth-surface-temperature-
data/GlobalLandTemperaturesByMajorCity
.csv")
GlobalLandTemperaturesByState = pd.rea
d_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByState.csv
")
GlobalTemperatures = pd.read_csv("../i
nput/climate-change-earth-surface-
temperature-
data/GlobalTemperatures.csv")

print(GlobalLandTemperaturesByCountry)
print(GlobalLandTemperaturesByCity)
```

Анализ данных из датасета:

```
import numpy as np # linear algebra
# data processing, CSV file I/O (e.g. p
d.read_csv)
import pandas as pd
# Input data files are available in the
 "../input/" directory.
# For example, running this (by clickin
g run or pressing Shift+Enter)
# will list all files under the input d
irectory
import os

for dirname, _, filenames in os.walk('/
kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, fil
ename))
# Any results you write to the current
directory are saved as output.
import pandas as pd

GlobalLandTemperaturesByCity = pd.read_
csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCity.csv")

GlobalLandTemperaturesByCountry = pd.re
ad_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCountry.cs
v")
GlobalLandTemperaturesByMajorCity = pd.
read_csv("../input/climate-change-
earth-surface-temperature-
```

```
data/GlobalLandTemperaturesByMajorCity.
csv")
GlobalLandTemperaturesByState = pd.read
_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByState.csv"
)
GlobalTemperatures = pd.read_csv("../in
put/climate-change-earth-surface-
temperature-
data/GlobalTemperatures.csv")

df = GlobalLandTemperaturesByCity
print(df.describe())
```

Вывод всех стран, которые есть в датасете:

```
import numpy as np # linear algebra
# data processing, CSV file I/O (e.g. p
d.read_csv)
import pandas as pd
# Input data files are available in the
 "../input/" directory.
# For example, running this (by clickin
g run or pressing Shift+Enter)
# will list all files under the input d
irectory
import os

for dirname, _, filenames in os.walk('/
kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, fil
ename))
# Any results you write to the current
directory are saved as output.
import pandas as pd

GlobalLandTemperaturesByCity = pd.read_
csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCity.csv")
GlobalLandTemperaturesByCountry = pd.re
ad_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCountry.cs
v")
GlobalLandTemperaturesByMajorCity = pd.
read_csv("../input/climate-change-
earth-surface-temperature-
data/GlobalLandTemperaturesByMajorCity.
csv")
GlobalLandTemperaturesByState = pd.read
_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByState.csv"
)
GlobalTemperatures = pd.read_csv("../in
put/climate-change-earth-surface-
temperature-
data/GlobalTemperatures.csv")

print(pd.unique(df['Country']).tolist()
)
print(f"Number of countries = {len(pd.u
nique(df['Country']).tolist())}")
```

Индивидуальное задание

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy import interpolate

import numpy as np # linear algebra
# data processing, CSV file I/O (e.g. pd
.read_csv)
import pandas as pd
# Input data files are available in the
"../input/" directory.
# For example, running this (by clicking
 run or pressing Shift+Enter)
# will list all files under the input di
rectory
import os

for dirname, _, filenames in os.walk('/k
aggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, file
name))
# Any results you write to the current d
irectory are saved as output.
import pandas as pd

GlobalLandTemperaturesByCity = pd.read_c
sv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCity.csv")
```

```
GlobalLandTemperaturesByCountry = pd.rea
d_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByCountry.csv
")
GlobalLandTemperaturesByMajorCity = pd.r
ead_csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByMajorCity.c
sv")
GlobalLandTemperaturesByState = pd.read_
csv("../input/climate-change-earth-
surface-temperature-
data/GlobalLandTemperaturesByState.csv")

GlobalTemperatures = pd.read_csv("../inp
ut/climate-change-earth-surface-
temperature-
data/GlobalTemperatures.csv")


df = GlobalLandTemperaturesByCity
# преобразуем строку в тип данных дата-
время
df['dt'] = df['dt'].astype('datetime64[n
s]')
# группируем по стране и по году в дате
df_by_dt = df.groupby([df['Country'],df[
'dt'].dt.year]).mean()
# убираем индексы, Country и dt становят
ся column
df_by_dt = df_by_dt.reset_index()


df = df_by_dt
# запишем более коротко условие по выбор
у стран
df_by_country = df[df['Country'].isin(['
Russia', 'Italy', 'Germany'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).toli
st()

# преобразуем таблицу так, чтобы столбцы
 опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(
['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))
```

```
# выбираем только те столбцы, где есть с
толбец AverageTemperature
newcols = [item for item in dfun if item
[0]=='AverageTemperature']
# строим новую таблицу из столбцов
dpop = dfun[newcols]
# выводим наш график
dpop.plot()


df = GlobalLandTemperaturesByCountry
# преобразуем строку в тип данных дата-
время
df['dt'] = df['dt'].astype('datetime64[n
s]')
# группируем по стране и по году в дате
df_by_dt = df.groupby([df['Country'],df[
'dt'].dt.year]).mean()
# убираем индексы, Country и dt становят
ся column
df_by_dt = df_by_dt.reset_index()

df = df_by_dt
# запишем более коротко условие по выбор
у стран
df_by_country = df[df['Country'].isin(['
Africa'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).toli
st()

# преобразуем таблицу так, чтобы столбцы
 опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(
['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))

# выбираем только те столбцы, где есть с
толбец AverageTemperature
newcols = [item for item in dfun if item
[0]=='AverageTemperature']
# строим новую таблицу из столбцов
dpop = dfun[newcols]
# выводим наш график
dpop.plot()


df = df_by_dt
```

```
# запишем более коротко условие по выбор
у стран
df_by_country = df[df['Country'].isin(['
Asia'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).toli
st()

# преобразуем таблицу так, чтобы столбцы
 опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(
['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))

# выбираем только те столбцы, где есть с
толбец AverageTemperature
newcols = [item for item in dfun if item
[0]=='AverageTemperature']
# строим новую таблицу из столбцов
dpop = dfun[newcols]
# выводим наш график
dpop.plot()

df = df_by_dt
```

```
# запишем более коротко условие по выбор
у стран
df_by_country = df[df['Country'].isin(['
South America'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).toli
st()

# преобразуем таблицу так, чтобы столбцы
 опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(
['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))

# выбираем только те столбцы, где есть с
толбец AverageTemperature
newcols = [item for item in dfun if item
[0]=='AverageTemperature']
# строим новую таблицу из столбцов
dpop = dfun[newcols]
# выводим наш график
dpop.plot()
```

Код сглаживания графика:

```
import pandas as pd
import pandas
import matplotlib.pyplot as plt
from scipy import interpolate

import numpy

import numpy as np # linear algebra
# data processing, CSV file I/O (e.g. pd.read_csv)
import pandas as pd
# Input data files are available in the "../input/" directory.
# For example, running this (by clicking run or pressing Shift+Enter)
# will list all files under the input directory
import os

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
# Any results you write to the current directory are saved as output.
import pandas as pd
```

```
GlobalLandTemperaturesByCity = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCity.csv")
GlobalLandTemperaturesByCountry = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCountry.csv")
GlobalLandTemperaturesByMajorCity = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByMajorCity.csv")
GlobalLandTemperaturesByState = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByState.csv")
GlobalTemperatures = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalTemperatures.csv")


df = GlobalLandTemperaturesByCity
# преобразуем строку в тип данных дата-время
df['dt'] = df['dt'].astype('datetime64[ns]')
# группируем по стране и по году в дате
df_by_dt = df.groupby([df['Country'],df['dt'].dt.year]).mean()
# убираем индексы, Country и dt становятся column
df_by_dt = df_by_dt.reset_index()


df = df_by_dt
# запишем более коротко условие по выбору стран
df_by_country = df[df['Country'].isin(['Russia'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).tolist()

# преобразуем таблицу так, чтобы столбцы опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))

# выбираем только те столбцы, где есть столбец AverageTemperature
newcols = [item for item in dfun if item[0]=='AverageTemperature']
# строим новую таблицу из столбцов
print(dfun)
print(newcols)
dpop = dfun[newcols]
# выводим наш график
from scipy.ndimage.filters import gaussian_filter1d
import numpy as np
import matplotlib.pyplot as plt
import scipy

import seaborn as sns
from scipy.interpolate import interp1d

lastx = 0
summvar = 0
lastsummvar = 0
```

```
def weighted_mean(x, **kws):
    global lastx
    global summvar
    global lastsummvar

    lastsummvar=(np.sum(x)/np.sum(lastx))
    summvar = np.sum(x)-((lastsummvar))

    lastx = np.sum(x)
    return summvar

#seaborn.violinplot(y='dt', x='AverageTemperature', data=df_by_country)
seaborn.lineplot(x='dt',y='AverageTemperature', data=df_by_country, estimator=weigh
ted_mean, lw=20, sizes=(.25, 2.5) )
dpop.plot()
# выбираем только те столбцы, где есть столбец AverageTemperatureUncertainty
```

Изучение возможностей доступа к API сервиса data.gov.ru:

```
import http.client as cl
import ssl
import json
import pandas
import pandas
fname = '/content/drive/MyDrive/python/dataf/dataset.hdf5'
dt = pandas.read_hdf(fname, key = 'datasets_2019')
#print(dt)
#dt_filt = dt[dt['title'].str.contains(r'студент')]
#print(dt_filt[['title','identifier']])
conn = cl.HTTPSConnection('data.gov.ru', timeout=10000,
 context=ssl._create_unverified_context())
for i in range(20,100):
 print("identifier ",dt['identifier'].iloc[i])
 print("title ",dt['title'].iloc[i])
 # задаем запрос на получение идентификатора версии, который будем использовать
 # при получении структуры и контента датасета
 request = "https://data.gov.ru/api/json/dataset/"
 request = request + dt['identifier'].iloc[i]+"/version/?access_token=2ab2f3ca4602a
791d37173e4a6708aa5"
 conn.request("GET", request)
 resp = conn.getresponse()
 # берем json содержимой ответа
 js_data = json.load(resp)
 print("version ",js_data[0]['created'])
 ident_v = js_data[0]['created']
 # задаем строку запроса для получения структуры датасета
 request = "https://data.gov.ru/api/json/dataset/"
 request = request + dt['identifier'].iloc[i]+"/version/"
 request = request+ ident_v+"/structure?access_token=2ab2f3ca4602a791d37173e4a6708a
a5"
 conn.request("GET", request)
```

```
 resp = conn.getresponse()
 js_data = json.load(resp)
 # если при получении структуры данные не пустые, выводим данные о структуре
 if js_data:
  print(js_data)
 # делаем запрос на получение данных в формате xml
 request = "https://data.gov.ru/api/xml/dataset/"
 request = request + dt['identifier'].iloc[i]+"/version/"
 request = request+ ident_v+"/content?access_token=2ab2f3ca4602a791d37173e4a6708aa5
"
 conn.request("GET", request)
 resp = conn.getresponse()
 data_content = resp.read().decode()
 print(data_content[0:24])
 # попробуем обработать наши данные и парсить xml строку
 import pandas as pd
 import xml.etree.ElementTree as et
 import xml.dom.minidom
 # используем стандартный парсер Python, есть и сторонние
 # Два основных парсера DOM и sax. Почитайте, что это.
 parser = et.XMLParser(encoding="utf-8")
 xroot = et.fromstring(data_content, parser=parser)
 # ищем в xml теги, используя структуру, предлагаемую API
 print(xroot.tag)

conn.close()
```

7. Дополнительные Задания

1) Нарисовать графики температур по США, Германии , Франции на одном графике

2) Нарисовать графики отношений температур этих стран

1 Задание

Код программы:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy import interpolate

import numpy as np # linear algebra
# data processing, CSV file I/O (e.g. pd.read_csv)
import pandas as pd
# Input data files are available in the "../input/" directory.
# For example, running this (by clicking run or pressing Shift+Enter)
# will list all files under the input directory
import os

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
# Any results you write to the current directory are saved as output.
import pandas as pd

GlobalLandTemperaturesByCity = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCity.csv")
GlobalLandTemperaturesByCountry = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCountry.csv")
GlobalLandTemperaturesByMajorCity = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByMajorCity.csv")
GlobalLandTemperaturesByState = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByState.csv")
GlobalTemperatures = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalTemperatures.csv")

df = GlobalLandTemperaturesByCity
# преобразуем строку в тип данных дата-время
df['dt'] = df['dt'].astype('datetime64[ns]')
# группируем по стране и по году в дате
df_by_dt = df.groupby([df['Country'],df['dt'].dt.year]).mean()
# убираем индексы, Country и dt становятся column
df_by_dt = df_by_dt.reset_index()

df = df_by_dt
# запишем более коротко условие по выбору стран
df_by_country = df[df['Country'].isin(['France', 'United States', 'Germany'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).tolist()

# преобразуем таблицу так, чтобы столбцы опять стали индексами
# либо уберем reset_index() в коде выше
df_by_country = df_by_country.set_index(['Country','dt'])
dfun = df_by_country.unstack('Country')
# печатаем полученные столбцы
print(list(dfun))
```

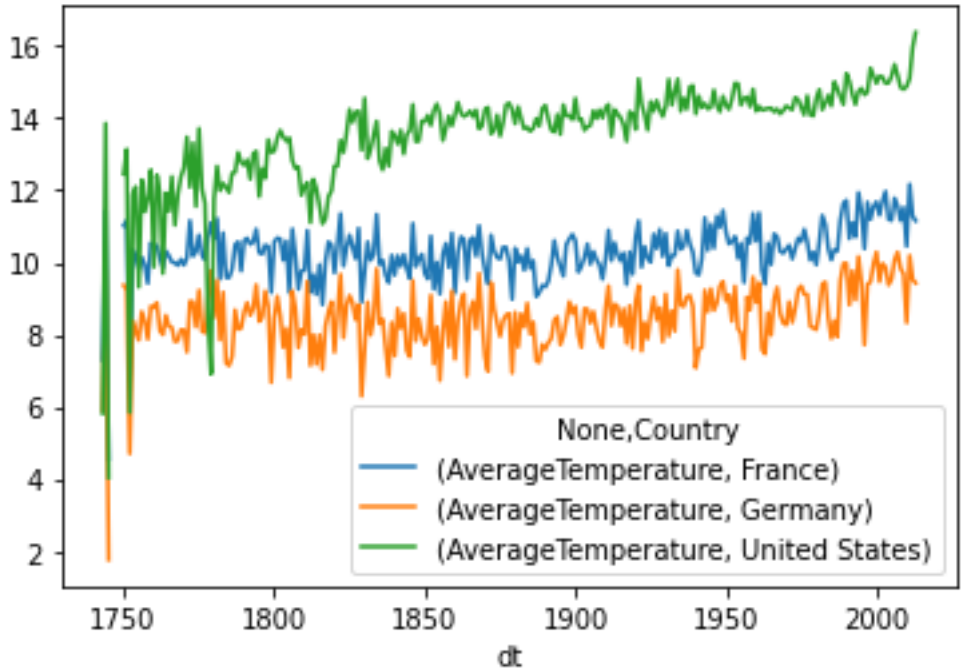


Рисунок 7.1 — результат работы кода 1 дополнительного задания

2 Задание

Код программы:

```
import numpy as np
import pandas as pd
import plotly.express as px
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

GlobalLandTemperaturesByCity=pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCity.csv")
GlobalLandTemperaturesByCountry=pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByCountry.csv")
GlobalLandTemperaturesByMajorCity=pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByMajorCity.csv")
GlobalLandTemperaturesByState=pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalLandTemperaturesByState.csv")
GlobalTemperatures = pd.read_csv("../input/climate-change-earth-surface-temperature-data/GlobalTemperatures.csv")
df = GlobalLandTemperaturesByCountry
# преобразуем строку в тип данных дата-время
df['dt'] = df['dt'].astype('datetime64[ns]')
# группируем по стране и по году в дате
df_by_dt = df.groupby([df['Country'],df['dt'].dt.year]).mean()
# убираем индексы, Country и dt становятся column
df_by_dt = df_by_dt.reset_index()

df = df_by_dt
# запишем более коротко условие по выбору стран
df_by_country = df[df['Country'].isin(['France', 'United States', 'Germany'])]
print(df_by_country.columns.tolist())
print(df_by_country)
pd.unique(df_by_country['Country']).tolist()

import seaborn as sns
print(list(df))
```

```
sns.barplot(x = 'Country', y = 'AverageTemperature', data = GlobalLandTemperaturesByCountry,order = ['France', 'United States', 'Germany'])
```

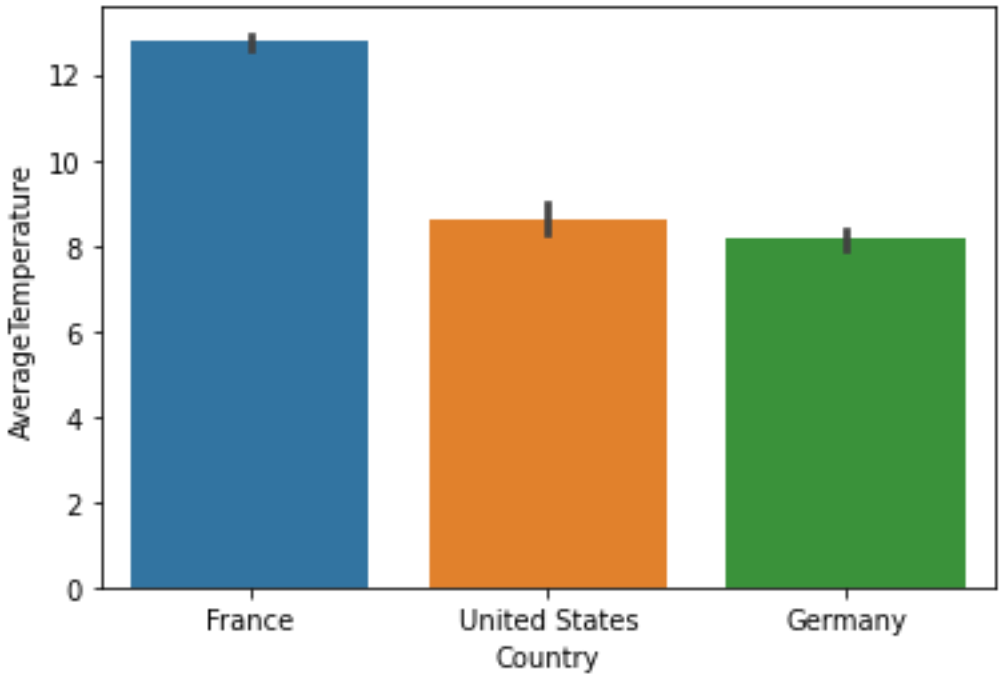


Рисунок 7.2 — результат работы кода 2 дополнительного задания